

Repository File Summary

This guide explains what every tracked file in this repo does.

.gitignore

This file tells Git which folders/files should NOT be saved in version history.

In this project, it ignores backend model cache data, Python virtual environment files, and Python cache files.

That is important because those files are usually huge or machine-specific.

Without this file, your repo can become very messy and slow.

When you edit this file:

- Add things that are generated automatically.
- Do not add source code files you actually want to track.

README.md

This is the main project manual.

It explains what the app does, how to run it, how backend mode works, and what each mode means.

It also includes developer notes, API behavior, and a flow diagram for optimize requests.

If someone joins the project, this is the first file they should read.

If product behavior changes, README should be updated too.

index.html

This is the main web page structure.

Think of it as the app skeleton.

It defines all visible UI pieces like:

- title and hero section
- model status card
- controls (engine, runtime, backend URL, mode toggles)
- code input/output panes
- score cards and timing info
- status pills (backend/model/fallback)

It does not run logic by itself.

It creates placeholders and IDs that JavaScript uses.

package.json

This is the Node project control file.

It stores project metadata and commands you can run with npm.

In this repo, it mainly defines scripts like:

- start frontend server
- start backend API
- set up backend environment
- download backend model files

So if you ask “how do I run this app?”, this file contains the real commands.

package-lock.json

This file locks exact npm dependency versions.

Right now this project is mostly static frontend, so this file is very small.

It still matters for reproducibility.

Why it matters:

- same dependencies on every machine
- fewer “works on my machine” issues

Usually you do not edit this file by hand.

It is auto-managed by npm.

src/main.js

This is the brain of the frontend.

It contains almost all app behavior and user interaction logic.

Major jobs of this file:

- Reads user inputs from the page.
- Handles engine choice (browser model vs backend API).
- Loads browser models (Phi-3 or TinyLlama) when needed.
- Calls backend `/optimize` when backend mode is selected.
- Applies timeout handling so backend calls do not hang forever.
- Parses model output and normalizes it.
- Falls back to local heuristic optimization when model output is unavailable.

- Updates UI with:
- optimized code
- change list
- quality/impact scores
- timing breakdown
- source/model labels

In simple terms:

HTML is the body, CSS is the look, and this file is the brain and nervous system.

src/styles.css

This file controls all visual styling.

It decides layout, spacing, fonts, colors, button style, cards, and responsive behavior.

Major jobs:

- Defines design tokens (colors, shadows, spacing feel).
- Styles hero, controls, editor panels, and score cards.
- Styles guidance strip and status chips.
- Handles mobile/tablet breakpoints for responsive UI.

In simple terms:

If the app “looks good” or “looks broken,” this file is usually where the reason lives.

backend/app.py

This is the backend API server (FastAPI).

It receives optimize requests from the frontend and returns optimization results.

Main endpoints:

- `GET /health`: basic alive check.
- `GET /status`: runtime diagnostics (model readiness, cache info, auto-selection reason).
- `POST /optimize`: main optimization route.

What happens in optimize:

1. Backend checks if requested model is ready.
2. If strict mode requires model and model is missing, it returns an error.
3. If model is available, it runs model inference.

4. If model fails and strict mode is off, it uses safe local heuristic fallback.
5. It always returns metadata (source label, model label, timings).

This file is where backend truthfulness and reliability are enforced.

backend/download_models.py

This is a helper script to download and cache model files once.

It stores models in `backend/model-cache/<model\_key>`.

It supports downloading one model or all models, and force re-download.

It also writes a marker file after success and validates key model artifacts.

That helps backend know if a model cache is truly usable.

If backend model inference is failing because files are missing, this file is part of the fix path.

backend/requirements.txt

This file lists core Python packages needed for backend API operation.

It includes things like FastAPI and Uvicorn, plus Hugging Face Hub client.

Think of this as the “minimum backend packages to run the service.”

backend/requirements-models.txt

This file lists extra heavy packages needed for local model runtime.

It includes torch, transformers, safetensors, and related pieces.

Use this when you want actual local model inference.

If you only need fallback/heuristic backend behavior, this may not be required in all setups.

Quick mental model (very simple)

- `index.html` = page structure
- `src/styles.css` = page look and style
- `src/main.js` = frontend logic and behavior
- `backend/app.py` = backend API brain
- `backend/download\_models.py` = model download helper
- `README.md` = user/developer instructions
- `package.json` = run commands

- `requirements*.txt` = Python dependency lists
- `.gitignore` = keeps junk out of git
- `package-lock.json` = locks exact npm dependency state

If you are explaining the repo to a non-technical person:

This is a web app with a frontend and a Python backend.

Frontend collects code and shows results.

Backend does optimization (model-based if available, safe fallback if not).

Everything else supports setup, styling, and reliability.